6.033 Spring 2018

Lecture #14

- **Reliability via Replication**
  - **General approach to building fault-tolerance systems**
  - **Single-disk failures: RAID**

1

6.033 | spring 2018 | Katrina LaCurts

这一讲的主线：用不可靠组件构建可靠服务

这是本讲的路线图：先学习怎样系统地分析故障，再用磁盘阵列说明冗余如何提高可靠性。这里的目标不是让每块硬件永不损坏，而是在约定的故障范围内仍维持服务。

1. 把“组件坏了”与“服务失败”分开

一块盘失效是组件事件；如果另一份数据仍可读取，用户服务未必立即失败。这种分离正是容错（fault tolerance）的价值。

```
# 自编思考框架
component fails -> detect -> recover/mask
question: does the user-visible service still work?
```

2. 三层问题要一起问

先问会发生什么故障，再问怎样发现、隔离和处理；最后计算为此多用了多少存储、时间与实现复杂度。只说“多复制一份”没有说明保证的前提。

3. 本讲的范围

本讲先处理单机内的磁盘故障。跨机器复制、事务等内容在官方提纲中作为后续课程方向出现，不能把它们当成这几张 RAID 图已经解决的问题。

> **记住：可靠服务是一项系统属性；增加副本之前，先定义要承受的故障。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



第一步：先写清楚系统要面对哪些故障

课件先强调 identify：识别可能故障。它要求设计者建立故障模型，而不是等系统坏了再临时猜测。配套官方提纲列举了软件、硬件、设计、操作和环境等来源。

1. 故障模型决定方案有没有用

若假设只有一块磁盘完全失效，镜像或单校验方案可能合适；若假设同一电源会带走整个阵列，仅在阵列内加盘就不够。先写前提，才能解释保证。

自编场景	需要明确的假设
一块盘损坏	其他盘仍正确、可读
整个机柜断电	副本是否共用同一电源
程序错误写入	副本是否会一起收到错误数据

2. 列表是分析工具，不是完美预言

“识别所有故障”是设计目标，并不表示我们已经知道一切。官方提纲强调过程需要迭代；每次故障或测试结果都可能暴露遗漏。

3. 从现象往模型追问

超时、错误结果、无法启动属于可见现象。实际原因可能不同，所以要区分观察到的证据、提出的假设和已经验证的根因。

> **记住：不写故障假设，就无法判断一个容错结论覆盖了什么。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



蓝屏是故障现象，不是完整的根因诊断

这张旧版 Windows 错误画面承接上一页：现实系统会出现软件异常和终止。屏幕文字提到异常、应用终止以及继续或重启的操作选项。它不是一段算法。

1. 画面能告诉我们什么?

它表明某个异常已被暴露给用户，并且正常执行受到影响。但仅靠这张图，不能断言一定是内存硬件、磁盘或某个具体驱动坏了。

```
# 自编诊断记录
observed: exception reported, execution interrupted
unknown: root cause, affected data, safe recovery
next: inspect evidence before choosing a remedy
```

2. 检测不等于处理完毕

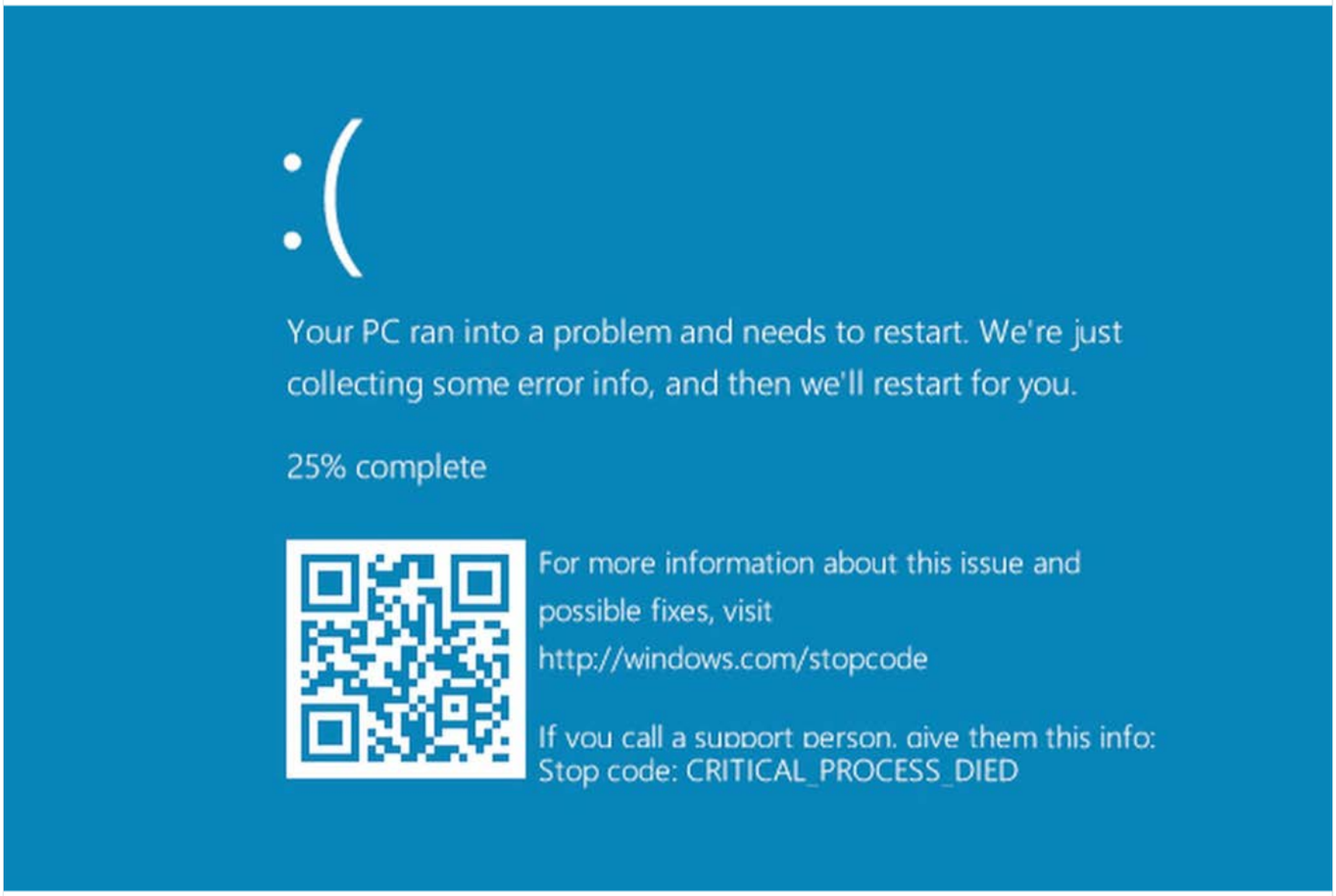
显示报错说明系统做了某种检测或报告；它没有自动证明数据完整、服务恢复，或者故障被成功限制在一个组件内。

3. 回到设计者的问题

需要哪些日志定位问题？异常能否只终止一个组件？重启后如何确认状态一致？这些是从截图引出的自编诊断问题，不是课件提供的确定答案。

> **记住：错误界面提供观察证据；根因、影响范围和恢复结果还要分别验证。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



新版蓝屏：错误收集、重启与服务恢复是不同阶段

画面显示问题报告、错误信息收集进度、停止代码和重启提示。它展示了故障处理的用户可见过程，并不说明重启后所有工作都会恢复。

1. 进度条在量什么?

25% 是画面中的错误信息收集进度，不是系统可靠性、数据恢复率或“25% 的问题已修好”。解释图像时要先确认数值的对象。

```
# 自编阶段划分
collect diagnostics -> restart -> check service/data
25% belongs to diagnostics, not recovery correctness
```

2. 停止代码只是线索

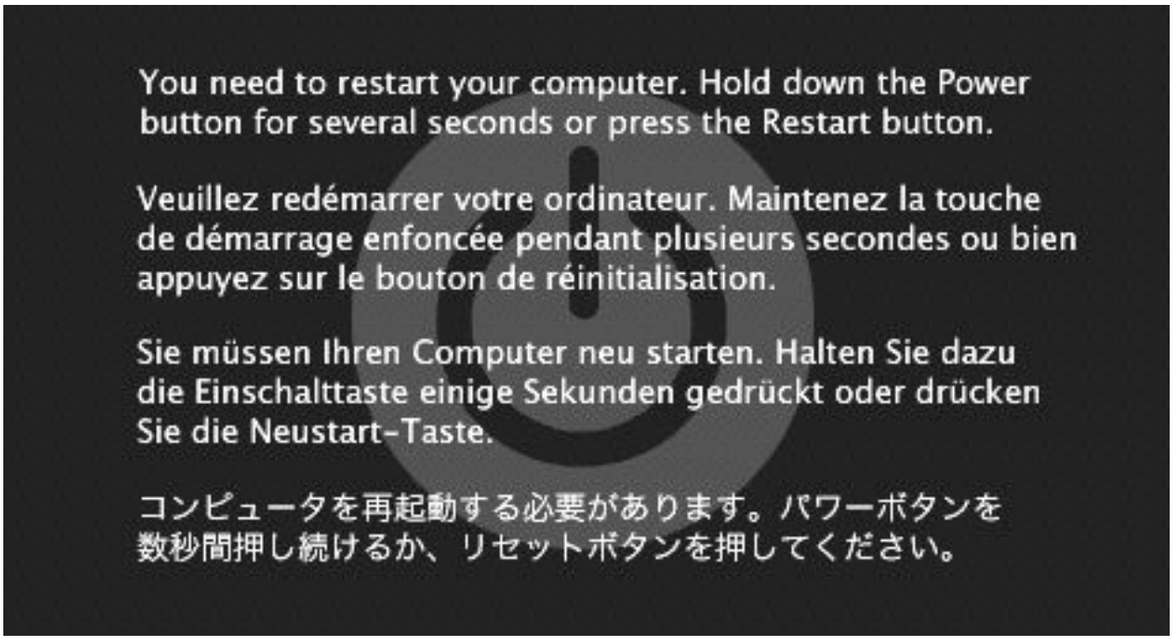
CRITICAL_PROCESS_DIED 可以帮助缩小调查方向，但从单张截图仍不能推出唯一根因。对症状分类与证明原因是两件事。

3. 重启还不是完整恢复

系统重新启动后，仍需检查服务是否可用、未完成的请求如何处理、数据是否一致。恢复时间也不仅是按下重启按钮的时间。

> **记住：收集错误信息、重启系统、恢复用户服务，需要分别确认。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



5

不同操作系统都会失败：冗余也要考虑共同原因

这张多语言重启提示与前两张 Windows 画面形成对照：更换操作系统品牌，并不会让故障这个工程问题消失。

1. 先读图，不凭图片补故事

画面要求用户重新启动。它没有给出完整日志，因此不能凭它判断具体代码路径、硬件损坏位置或丢失了哪些数据。

2. 重启有什么作用?

重启可能清除某些暂态状态并恢复运行，但不保证修复软件缺陷或硬件损坏。若相同条件再次触发，系统仍可能再次失败。

自编对照	可能结果
临时状态问题	重启后可能恢复
持久硬件故障	重启后仍可能失败
各副本共有的软件缺陷	多个副本可能同时失效

3. 副本会不会一起失败?

由这组截图可以引出一个自编边界：如果多个副本运行同样的有缺陷程序，并接收同样的输入，它们可能一起失败。复制数量多，不代表故障互相独立。

> **记住：冗余是否有效，要看副本能否避开同一种故障，而不仅是副本数量。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



容错的三步：识别、检测与限制、处理

这页把前面的铺垫收束成完整流程：identify，detect and contain，handle。三步分别回答“可能坏什么”“怎么发现并限制影响”“发现后做什么”。

1. 检测与限制影响

检测是发现不符合预期的状态；containment 是把故障影响限制在合适边界内。检测到一个组件异常，却让它继续破坏其他组件，不算完成隔离。

2. 处理并不总是掩盖错误

配套官方提纲列举了不处理、fail-fast、fail-stop 和 masking 等选择。报告错误并停止，有时比带着不可信状态继续返回结果更合适。

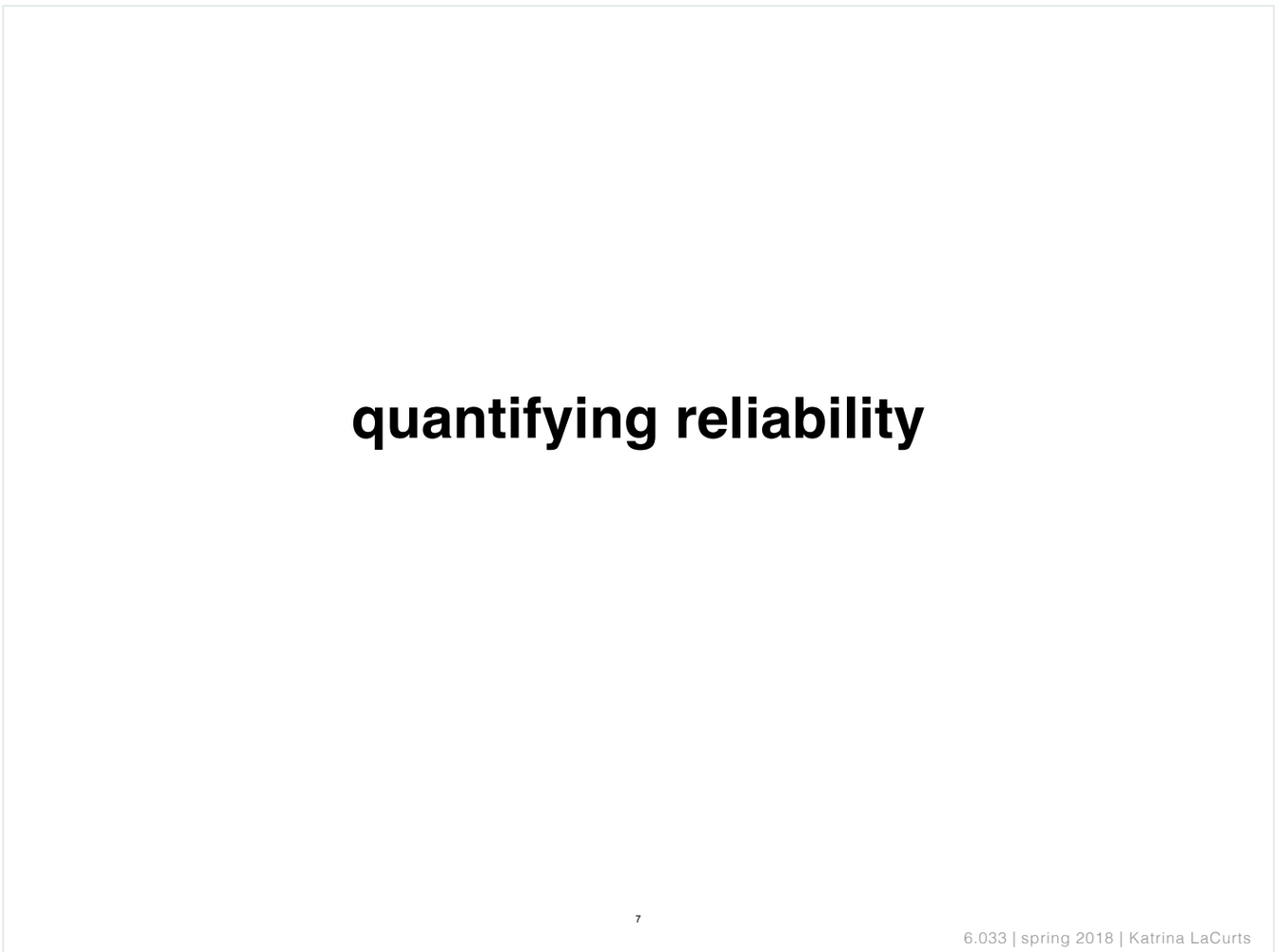
```
# 自编流程，非可执行程序
identify fault model
if abnormal state detected:
    contain its effects
    report / stop / recover / mask
```

3. 方法要跟应用需求匹配

自编例：数据校验失败后拒绝返回错误结果，是一种保护边界；读取健康镜像并继续服务，则是在指定故障模型下掩盖故障。不能假定“继续运行”永远最好。

> **记住：发现故障只是中间一步；还要限制影响，并选择满足服务语义的处理方式。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



量化可靠性：平均正常时间与平均修复时间

原页只有 quantifying reliability 标题。下面的指标与例子依据官方配套 Lecture 14 Outline 补充，不能说成原幻灯片上直接印出的公式。

1. 先统一指标与单位

本讲用 MTTF 表示平均正常运行到失效的时间，MTTR 表示平均修复/恢复时间。在反复正常运行与修复的简化模型下，可用性是正常时间占总时间的比例。

2. 手算官方提纲的例子

若每 30 天发生一次崩溃，每次恢复 10 分钟，则正常时间取 30×24×60=43,200 分钟。可用性为 43,200/(43,200+10)≈99.9769%。提纲粗略记为 .9997；按给定数字复算，保留四位小数应为 .9998。

```
# 提纲例子的复算；单位均为 minutes
MTTF = 30 * 24 * 60 = 43200
MTTR = 10
A = MTTF / (MTTF + MTTR)
  = 43200 / 43210 = 0.99976857
```

3. 提高比例有两条路

增大平均正常时间，或缩短平均恢复时间，都能改善这个比例。这个长期平均指标不等于“未来永不失败”，也不直接保证保存的数据不会丢失。

记住：单位先统一；可用性同时取决于多久坏一次，以及坏了多久能恢复。

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



为什么先拿磁盘故障做例子?

这是一张转场标题页。配套提纲解释：先在单机内讨论磁盘，建立处理组件故障的思路，再扩展到多机器系统。

1. 磁盘与其他组件的区别

更换 CPU 之后，已有数据通常还在存储上；磁盘失效则可能让唯一的数据副本一起丢失。因此问题不只是恢复计算能力，还包括保存持久状态。

恢复对象	需要确认什么
硬件可运行	替换组件能否工作
数据仍存在	是否有副本或可用校验
用户服务恢复	系统状态与应用语义是否正确

2. 把服务恢复与数据恢复分开

自编例：换上一块空白新盘能让硬件再次启动，但不会自动找回原来的文件。需要事先保存的副本、校验信息或备份才能恢复数据。

3. 明确本讲的简化范围

随后主要讨论整盘失效：一块盘上的所有块不可用。静默数据损坏、误删除、机房级共同故障等问题不能直接由单盘恢复结论覆盖。

> **记住：换回硬件不等于找回数据；容错存储必须提前保存恢复所需的信息。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义

Public adaptation / original page 9

Original vendor image omitted: it is explicitly excluded from the OCW Creative Commons license.

Teaching point retained: a quoted MTBF is not an individual lifetime guarantee.

700,000 hours / (24 x 365) = 79.91 years

Illustrative aggregate estimate from the official lecture outline:

1,000 disks x 3,000 hours / 4 failures = 750,000 device-hours per failure

The aggregate example is hypothetical; it is not a claim about the vendor's actual test.

View the original image in the official PDF linked in examples/source.json.

Source lecture: Katrina LaCurts / MIT 6.033 / Spring 2018 / original p.9

700,000 小时：群体统计，不是某块硬盘的寿命保证

原页展示厂商规格表，并将 700,000 小时换算成约 80 年。该厂商图片明确排除在开放许可外，所以公开版在保留第 9 页位置的同时，以文字示意替换图片；原图可从官方原版查看。

1. 换算正确，也可能解读错误

700,000/(24×365)≈79.9 年只是单位换算。不能因此推出买来的某块硬盘保证使用 80 年；群体统计指标、单体寿命与保修承诺是不同概念。

2. 这个大数字可能怎样得到?

官方提纲给了一个假设例：1,000 块盘各观察 3,000 小时，总计 3,000,000 盘小时；若有 4 次故障，可估算为每 750,000 盘小时一次故障。这是解释统计口径的例子，不是对该厂商真实测试方法的断言。

```
# 官方提纲中的假设示例；不是厂商测试记录
total_device_hours = 1000 * 3000 = 3000000
failures = 4
rate_estimate = 4 / 3000000 per device-hour
time_per_failure = 3000000 / 4 = 750000 hours
```

3. 年龄与样本条件也重要

如果故障风险随设备年龄变化，或样本与部署环境不同，就不能直接把该估计当成每个时刻恒定的失效率。MTBF 的口径也不能未经定义就与 MTTF 或寿命混用。

记住：大 MTBF 数字需要连同样本、观察窗口和故障模型一起解释。

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



整盘故障与年龄相关风险：先明确要恢复什么

课件再次以 dealing with disk failures 转场到整盘恢复方案。配套提纲在这一段强调：失效风险并不一定无记忆，设备寿命早期和晚期可能更容易出现故障。

1. 不要把平均值当成恒定风险

年龄相关风险常用“浴盆形”直觉理解：早期缺陷、较稳定阶段、老化阶段。这里是配套提纲的定性说明，不是这张标题页上绘出的实测曲线。

2. 分析时先写整盘失效模型

接下来假设一块盘整体不可读取，其他盘及其保存的信息正确可用。RAID 1、4、5 都是在这个简化范围内讨论恢复，并不是对任意损坏模式的承诺。

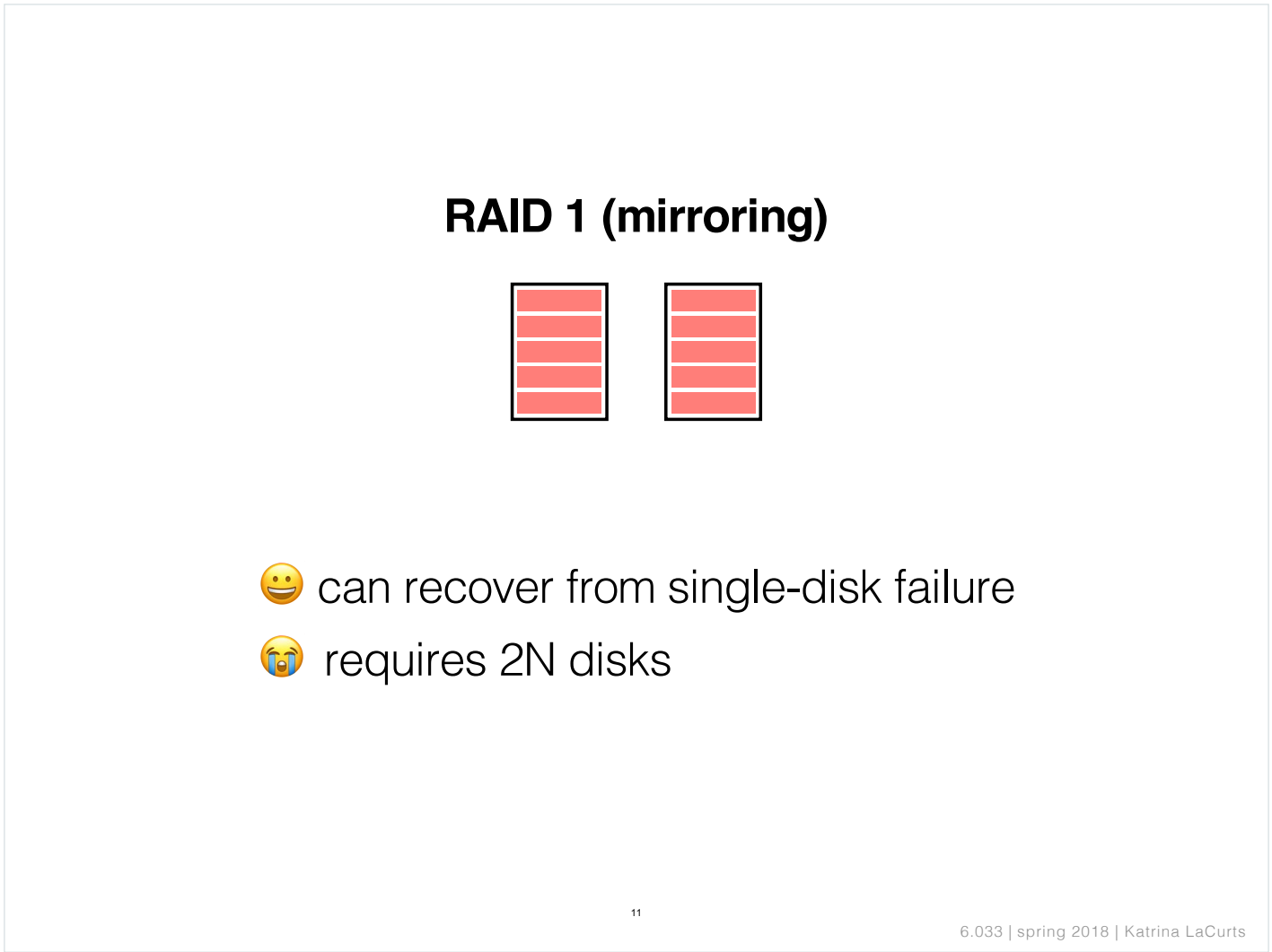
```
# 自编恢复条件
missing: all blocks on one failed disk
needed: readable survivors + enough redundancy
not assumed: all failures are independent or identical
```

3. 为恢复预留什么?

如果一条数据完全消失，要恢复它，系统必须仍有另一份内容或足够的独立校验关系。恢复能力来自保留的信息，不能在故障后凭空生成。

> **记住：先定故障粒度与存活条件，再问剩余信息是否足够恢复。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



RAID 1：用镜像保留第二份数据

图中两块盘的红色条带相同，表示同一份逻辑数据保存两份。镜像（mirroring）增加的是冗余，不是两倍的可用空间。

1. 为什么坏一块盘还能读?

假设两盘原本内容一致、另一盘仍可用，一块物理盘失效后，可以从另一块读出数据，再给替换盘复制。图中的 single-disk failure 指这种硬件故障。

```
# 自编镜像布局；同名块表示相同内容
disk_1: [A, B, C]
disk_2: [A, B, C]  # 完整副本
disk_1 fails -> read disk_2
```

2. 2N 到底在数什么?

这里 N 是存放原始数据需要的等容量盘数。每份再配一份镜像，因此要 2N 块物理盘。用两块 4 TB 盘保存一份数据时，可用容量仍为 4 TB，不能把副本重复算成新数据空间。

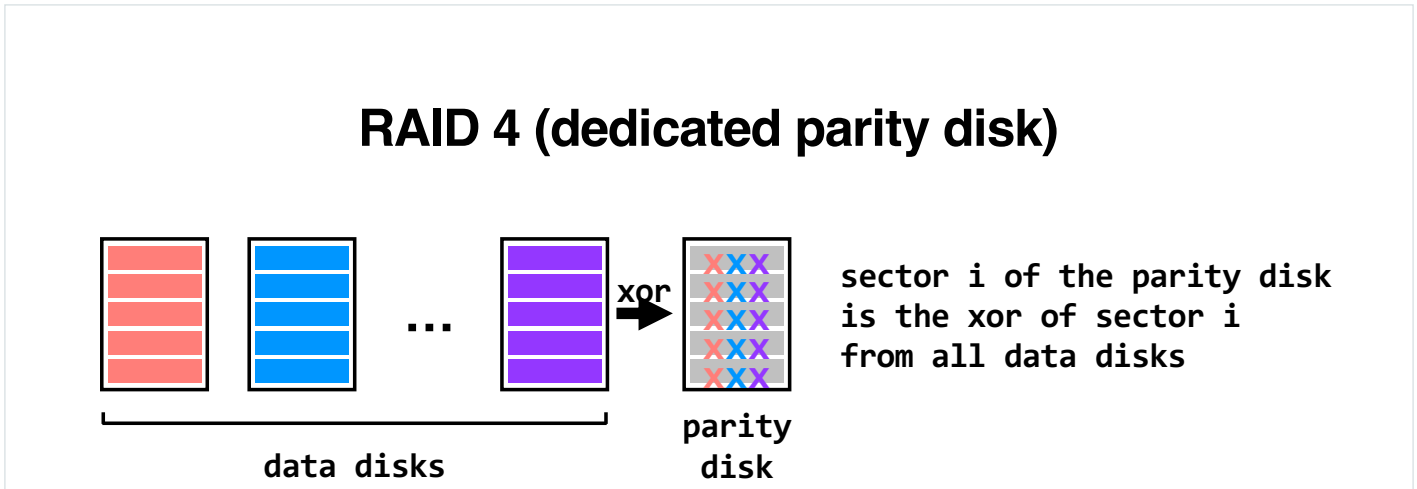
两盘镜像（自编例）	容量或含义
物理总容量	2 × 4 TB = 8 TB
可用数据容量	1 × 4 TB = 4 TB
额外一份副本	用于恢复，不是额外的逻辑数据空间

3. 哪些问题没有因此解决?

若同一镜像对的两盘都坏，仍会丢数据；误删除也可能同步到镜像。镜像帮助抵抗磁盘失效，不能代替保留历史版本的独立备份。

> **记住：镜像保存完整副本；容错的代价是约一半的原始容量用于冗余。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



RAID 4：用 XOR 校验代替完整复制

左边是 N 块数据盘，右边是一块专用校验盘。箭头表示：每个相同位置 i 的数据块做按位 XOR，得到校验块 P。它不是把所有数据串接到一块盘。

1. 怎样从校验恢复?

下面用两位二进制做自编恢复例。若 B 丢失，用存活的 A 与校验 P 再做 XOR，就能恢复 B。利用的是相同贡献相互抵消：x XOR x=0、x XOR 0=x。

```
# 自编：两位二进制；XOR 不是普通加法
A = 10
B = 11
P = A XOR B = 01
B_missing = A XOR P = 11
```

2. 为什么只多一块盘?

N 块数据盘加一块校验盘，总计 N+1 块。每条带只有一个独立校验关系，能解出一个未知块；同一条带丢两个块时，通常无法唯一恢复。

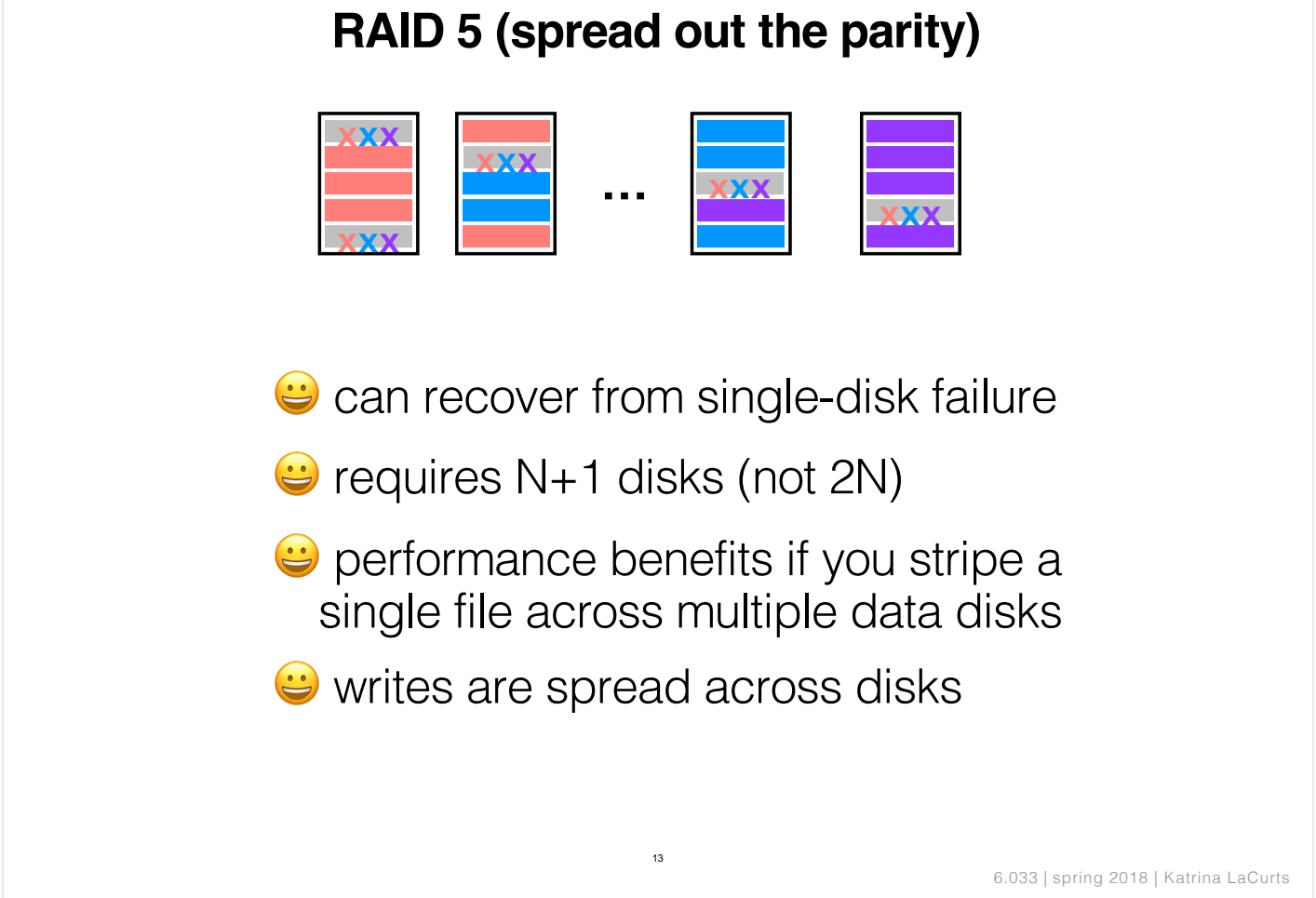
符号或状态	对应含义
N	存放原始数据需要的等容量盘数
N+1	数据盘与专用校验盘的物理总数
一个未知块	可由一份校验与其他正确块恢复

3. 读写性能的取舍

条带化（striping）把文件块分散到数据盘，合适负载可并行读。但每次正常数据更新还要更新同一块校验盘，使它成为共享写入瓶颈。这里不是说每次写都只写校验盘。

> **记住：一块校验盘节省空间、帮助恢复一块坏盘，也集中了承担校验更新的写负载。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义



RAID 5：把校验块的负担分散到各盘

灰色且标有 XXX 的块表示校验。与上一页不同，各条带的校验放在不同磁盘：第一条带在左盘，后面的条带轮换到其他盘。不是给同一条带增加多份独立校验。

```
# 自编四盘布局；P 为校验，D 为数据块
          disk_1  disk_2  disk_3  disk_4
stripe_1 P1      D1a     D1b     D1c
stripe_2 D2a     P2      D2b     D2c
stripe_3 D3a     D3b     P3      D3c
```

1. 解决哪个瓶颈?

RAID 4 的校验更新全落到一块盘。RAID 5 分散不同条带的校验位置，让写入负载较均衡；一次数据更新仍要维护对应校验，不能理解成“写入不再需要校验”。

2. 哪些性质没有变化?

每条带仍只有一份 XOR 校验，因此仍是 N 块盘的数据加一块盘等量的校验，物理总数 N+1。可恢复任意单盘失效，但通常不能恢复同一条带的两个未知块。

四盘阵列（自编例）	结论
N = 3	4 块物理盘，可用容量约为 3 块盘
校验位置分散	分散不同条带的校验更新负载
每条带一份校验	一般不能唯一恢复两个未知块

3. 把容量和安全分开想

自编例：4 块等大的物理盘，N=3，可用容量约等于 3 块盘；校验分散不等于凭空多出一块盘的容量。误删、整阵列共同故障和恢复期间的额外故障仍要另行处理。

> **记住：RAID 5 改变校验放在哪里；单条带的校验信息量和单盘容错级别没有因此增加。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义

- Systems have faults. We have to take them into account and build reliable, **fault-tolerant systems**. Reliability always comes at a cost — there are tradeoffs between reliability and monetary cost, reliability and simplicity, etc.
- Our main tool for improving reliability is **redundancy**. One form of redundancy is **replication**, which can be used to combat many things including disk failures (important, because disk failures mean lost data).
- **RAID** replicates data across disks in a smart way: RAID 5 protects against single-disk failures while maintaining good performance.

可靠性总要付出代价：把本讲连成一条推理链

总结页强调三件事：系统会故障；冗余是提高可靠性的主要工具之一；RAID 用有组织的数据与校验布局保护单盘故障，但可靠性、成本与简单性之间存在取舍。

1. 比较要用相同的记号和假设

以 N 块等容量盘的原始数据为基准，RAID 1 需要 2N 块物理盘，RAID 4/5 需要 N+1 块。不能一会儿把 N 当数据盘数，一会儿又当物理总盘数。

方案	主要取舍
RAID 1	完整镜像，空间开销较高
RAID 4	节省空间，但专用校验盘集中写入
RAID 5	分散校验负载，仍只有单盘容错

2. 数副本还不够

两个副本共用电源、运行同一错误程序或遭遇同一操作失误时，可能同时失败。只有写出独立性等假设，简单的概率相乘才有意义。

```
# 自编概率例；只在同一时间窗口且独立时成立
p = 0.01
P(both mirrors fail) = p * p = 0.0001
shared cause -> cannot assume p * p
```

3. 怎样回答方案选择题?

先写故障模型和目标，再比较容量、读写负载、恢复方式、实现复杂度以及未覆盖风险。避免只说“RAID 5 最好”，因为不同负载和目标会改变取舍。

> **记住：给出方案时，把“解决什么、代价是什么、没解决什么”一起写清楚。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义

MIT OpenCourseWare
https://ocw.mit.edu

6.033 Computer System Engineering
Spring 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

出处、许可与复习定位

这是 MIT OpenCourseWare 的课程与使用条款页，不是新增技术概念。保留它能让读者追溯作者、课程、学期以及原材料使用条件。

1. 怎样区分三种信息?

本册上方是课件的公开适配版，下方为中文讲解；标题页中缺省的细节会注明来自官方配套提纲；数值变式、布局和诊断问题标为自编，不能当作官方答案。

2. 本册处理了哪些页?

按原顺序覆盖第 1-15 页，包括封面、截图、转场和许可页。第 9 页的厂商图在公开版被明确替换，未暗中裁切或把改图伪装成原图。

3. 复习时如何使用?

先用逐页册理解图和机制，再用考场册的目录、概念编号、英文短答与术语索引查找。任何考试的实际范围和纸质资料规则仍需以该考试公告为准。

> **记住：来源页码、补充依据和自编说明，也是可靠学习资料的一部分。**

CC BY-NC-SA 4.0 | 讲解与适配：fuxi-pdf | 非官方讲义